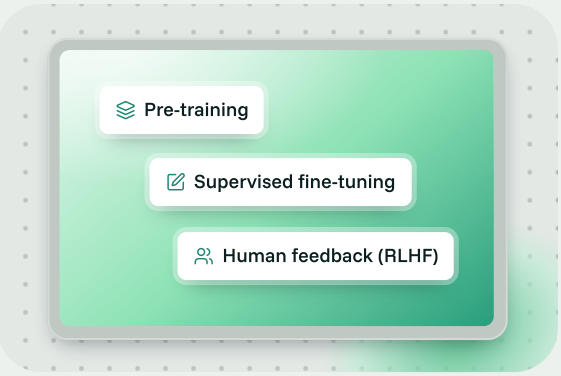
Last updated August 2026

What Is a Large Language Model?

A beginner-friendly guide to the machinery behind ChatGPT, Claude, Gemini, and Llama.

12 min read · Beginner-friendly



Pascal Academy
AI fundamentals · Part 1 of 10

What Is a Language Model?

A language model is, at its core, a probability engine for text. Given a sequence of words, it estimates how likely each possible next word is to appear.

That sounds simple, and the core idea genuinely is. A language model answers one question, over and over: given the text so far, what comes next? Everything you see from ChatGPT or Claude, whether it is writing code, summarising a document, or translating Hindi to English, is built on this single mechanism repeated billions of times.

That does not mean an LLM understands language in the way a human does. A model has never "had a thought" or experienced the world it is describing. What it has is a statistical sense of which words, ideas, and patterns tend to follow one another, learned from an enormous amount of text. That statistical sense turns out to be surprisingly powerful, powerful enough that the output can often look remarkably like understanding.

To see how this works, consider an incomplete sentence:

Example

When I hear rain on my roof, I _____ in my kitchen.

A language model assigns a probability to every possible token that could fill that blank. Some candidates are more likely than others, based on the patterns the model absorbed during training.

The table below shows what those probabilities might look like. The numbers are illustrative, not from a real model, but they capture the shape of what a language model produces every time it generates text.

Token(s) that could fill the blank	Probability	Likelihood
cook soup	9.4%	
warm up a kettle	5.2%	
cower	3.6%	
nap	2.5%	
relax	2.2%	

An application using this model can pick the highest-probability option (cook soup), or it can sample from the distribution, choosing randomly among the candidates weighted by their probabilities. Sampling is what gives models their variability. Two conversations with the same model rarely produce identical answers because the model is rolling weighted dice at each step.

Estimating what fills a blank is the primitive operation. From that single capability you get text generation (keep filling blanks), translation (fill in the foreign-language word), summarisation (fill in the short version), and code completion (fill in the next line of code). The task changes, the core mechanism does not.

Tokens and Probabilities

Before a language model can work with text, it needs to break that text into units called tokens. A token can be a whole word, a piece of a word, or sometimes a single character. The word tokenisation refers to the process of slicing text into these pieces.

English text tends to produce roughly one token per three-quarters of a word. The sentence 'I love cooking' might become three tokens: I, love, cooking. A longer word like uncharacteristically might be split into several subword tokens: un, character, istic, ally. Languages with non-Latin scripts often use more tokens per word, which is one reason inference costs can be higher for those languages.

Why not just use words?

If the model treated every full word as a unit, it would need a separate entry for run, running, runs, ran, and so on. Subword tokens let the model reuse the piece run across all of them, learning shared meaning and keeping the vocabulary manageable. The trade-off is that a single word can cost several tokens, which matters for pricing and context windows.

Once text is tokenised, the model's job is to assign probabilities over the vocabulary for the next token. The vocabulary of a modern model can be 50,000 to 200,000 tokens. At each step, the model computes a score for every token in that vocabulary and converts those scores into a probability distribution. The token with the highest probability is the model's top guess, but as mentioned, applications often sample from the distribution instead.

Parameters are the internal numbers the model uses to compute those probabilities. A 7-billion-parameter model has 7 billion learnable weights, each one a small dial that was tuned during training. More parameters generally means the model can represent more nuanced patterns, but it also means more compute, more memory, and more cost. Parameters alone do not determine quality, the training data and architecture matter just as much, but they are a rough proxy for a model's capacity.

Go deeper

How probabilities are computed

Optional

At each step, the model takes the entire sequence of tokens so far, passes them through its layers of computation, and produces a vector of raw scores called logits, one per token in the vocabulary. A function called softmax converts these logits into probabilities that sum to 1.0. The shape of the distribution is controlled by a parameter called temperature: at temperature 1.0 the distribution is unchanged, at lower temperatures it sharpens (favouring the top candidates more strongly), and at higher temperatures it flattens (making less likely tokens more probable). Temperature is one of the levers developers use to control how deterministic or creative a model's output feels.

N-grams and the Context Problem

The earliest language models counted word sequences. An n-gram is simply an ordered sequence of N words. When N is 2, you get a bigram, two words in a row. When N is 3, a trigram, three words in a row. The model builds a table of how often each n-gram appeared in its training corpus, then uses that table to guess the next word.

Given the phrase "you are very nice", the resulting bigrams are "you are", "are very", and "very nice". The trigrams from the same phrase are "you are very" and "are very nice". A bigram model that sees "orange is" looks up every bigram starting with "orange" and picks the most frequent follower. Similarly, a trigram model that sees "orange is" looks up trigrams starting with "orange is".

The problem with n-grams is that they capture very little context. Given "orange is", a trigram model can only look two words back. Is "orange" the fruit or the colour? The two-word window is not enough to tell. The model might predict "ripe" (fruit) or "cheerful" (colour), and it has no way to know which is correct without more surrounding text.

You could increase N to capture more context. A 5-gram looks back four words, a 7-gram looks back six. But as N grows, the number of possible sequences explodes. A 7-gram model trained on a large corpus will have seen most specific 7-word sequences only once, if at all. When every sequence is unique, the model has no statistical basis for prediction. The counts become too sparse to be useful.

This is the fundamental tension: short n-grams lack context, long n-grams lack data. Language models spent decades stuck in this trade-off.

Key takeaway

The n-gram approach teaches us something important: a good language model needs to use context efficiently. It cannot just memorise exact sequences, because most sequences it encounters at inference time will be novel. It needs to generalise from patterns it has seen. That insight is what drove the move to neural networks.

Neural Language Models

Recurrent neural networks, or RNNs, were the first serious attempt to break out of the n-gram trap. An RNN processes text token by token, and at each step it carries forward a hidden state, a kind of running summary of everything it has seen so far. The easiest way to understand it is to think of it as a reader who forms an impression as they go, updating their understanding with each new word.

This was a real improvement over n-grams. An RNN could, in principle, look back across an entire sentence or even a short paragraph. It could learn that "orange" followed by "is" followed by "ripe" is probably about fruit, and carry that forward to influence later predictions. The context window was no longer fixed at N minus one words.

But RNNs had two stubborn problems. First, they processed tokens sequentially. To predict the hundredth token, the model had to compute the first, then the second, then the third, all the way through. There was no way to parallelise this, which meant training on large datasets was slow. Second, and more fundamental, the hidden state tended to degrade over long sequences. Information from the start of a long passage would fade or become garbled by the time the model reached the end. This is the vanishing gradient problem, and it put a hard ceiling on how much context an RNN could usefully retain.

RNNs learn more context than n-grams, but the amount of useful context they can hold is still relatively limited.

LSTM and GRU architectures, popular in the 2010s, were clever workarounds that helped preserve information over longer distances. They helped, but they did not solve the core issue. Sequential processing was still the bottleneck, and truly long-range context remained out of reach.

What was needed was an architecture that could look at the entire input at once, not token by token, and weigh the relevance of each part to every other part. That architecture arrived in 2017.

Transformers and Self-Attention

In 2017, a team at Google published a paper called "Attention Is All You Need." The architecture it introduced, the Transformer, discarded the sequential processing of RNNs entirely. Instead of reading token by token, a Transformer looks at the entire sequence at once and decides which parts are relevant to which.

The mechanism at the heart of this is called self-attention and the name is apt: the model attends to itself, meaning every token in the input can look at every other token and decide how much weight to give each one. There is no fixed window or a sequential decay. The first word of a document can directly influence the last word, and vice versa.

In the sentence "the orange is ripe and sweet," the word "orange" is ambiguous on its own. But self-attention lets the model link "orange" directly to "ripe" and "sweet" without anything in between degrading the signal. The model learns to assign high attention weight between words that are relevant to each other, regardless of how far apart they sit in the sequence.

In an n-gram, the only context available is the last N minus one words. In an RNN, context has to survive a chain of hidden states. In a Transformer, every token has a direct line to every other token. That is why Transformers handle long-range dependencies so much better, and why they scale so well: the entire computation can be parallelised across the sequence, which makes training on massive datasets feasible.

Go deeper

How self-attention actually works

Optional

Each token is represented as three vectors: a query, a key, and a value. The query asks, what am I looking for? The key says, here is what I contain. The value is the actual information the token contributes. The model computes a dot product between each query and every key to produce attention scores, which are then normalised into weights. The output for each token is a weighted sum of all the values, where the weights come from those query-key matches.

Modern LLMs use multi-head attention, meaning they run this process many times in parallel. Each head can learn a different kind of relationship: one might focus on grammatical structure, another on semantic similarity, another on positional patterns. The outputs of all heads are concatenated and fed forward. This is why the architecture is so expressive, but also why it is computationally expensive: attention scales quadratically with sequence length, so doubling the context window quadruples the attention computation.

The "large" in large language model refers to scale: hundreds of millions to hundreds of billions of parameters, trained on trillions of tokens of text. The Transformer architecture is what made that scale possible, because it could be trained efficiently on the parallel hardware, GPUs and TPUs, that became available in the late 2010s. The combination of a better architecture and more compute is what produced the capability jump from early neural language models to the systems we use today.

Training an LLM

Building a large language model is a multi-stage process and each stage serves a different purpose, and understanding the pipeline helps explain why models behave the way they do.

Stage	Name	What Happens
1	Pre-training	Learn to predict the next token across trillions of words of internet text. The model absorbs grammar, facts, reasoning patterns, and style.
2	Supervised Fine-tuning	Train on curated examples of good question-answer pairs. The model learns the format and tone of helpful responses.
3	RLHF / RLAIF	Reinforcement learning from human (or AI) feedback. Humans rank model outputs by quality, and the model learns to prefer the better ones.
4	Alignment and Safety	Additional training to refuse harmful requests, reduce bias, and keep outputs within intended use. Often iterative and ongoing.

Pre-training

Pre-training is where the model learns the fundamentals. The training objective here is to simply predict the next token. Given a chunk of text, the model tries to guess what comes next, and when it gets it wrong, the error signal updates its parameters. Do this trillions of times across a massive corpus of internet text, books, code, articles, and the model develops a deep internal representation of language, facts, and reasoning patterns.

The model that comes out of pre-training is called a base model. It is not yet a chatbot. If you prompt a base model with "What is the capital of France?", it might respond with "What is the capital of Germany?" because it has learned to continue text, not to answer questions. It will happily complete a quiz, write an essay, or continue a story, but it does not yet know it is supposed to be helpful.

Supervised Fine-tuning

Supervised fine-tuning, or SFT, teaches the model the format of a helpful assistant. Developers create thousands of example conversations: a user asks a question, an assistant gives a good answer. The model is trained on these pairs, learning to produce responses in that format rather than just continuing the prompt. After SFT, the model knows to answer the question.

Reinforcement Learning from Human Feedback

RLHF, popularised by OpenAI with InstructGPT and later ChatGPT, adds a further layer. Human reviewers are shown multiple model outputs for the same prompt and asked to rank them by quality. A separate model, called a reward model, learns to predict these rankings. The main model is then fine-tuned to maximise the reward, meaning it learns to produce outputs that humans rate highly: helpful, honest, and harmless.

RLHF is what makes a model feel polished. It is also where a lot of the subjective behaviour of a model, its tone, its refusals, its tendency to be cautious or verbose, gets shaped. Different labs have different philosophies here, which is why Claude, GPT, and Gemini can feel quite different even though they share the same underlying architecture.



Go deeper

Why RLHF changed everything

Optional

Before RLHF, language models were impressive but difficult to use. They would continue text, but not necessarily in the direction you wanted. RLHF gave developers a way to steer models toward useful behaviour without rewriting the base model. The reward model encodes what humans prefer in an abstract sense, and optimising against it produces models that feel like they understand instructions. The catch is that the reward model is only as good as the human annotations it learns from, and annotators disagree, have biases, and can be inconsistent. This is one source of the ongoing challenges with alignment.

Key Problems

Large language models are powerful, but they are not perfect. Three problems in particular are worth understanding, because they shape how these models can and cannot be used.

Problem 1: Hallucination

Models generate text that sounds confident and specific but is factually wrong. A model might cite a paper that does not exist, give a wrong date for a historical event, or fabricate an API method. This happens because the model is optimised for plausible-sounding text, not for truth. It has no built-in fact-checker; it produces what is statistically likely given its training, and sometimes what is likely is also wrong.

Problem 2: Bias

Models learn from text written by humans, and human text carries biases. If the training corpus underrepresents certain groups or overrepresents certain viewpoints, the model will reflect that. A model might default to male pronouns for doctors, associate certain names with certain professions, or treat Western perspectives as universal. this is also not a bug you can patch because ultimately it is the property of the training data that requires deliberate mitigation.

Problem 3: Reasoning Limits

Models can appear to reason but often do so through pattern matching rather than genuine logical deduction. They struggle with multi-step arithmetic, novel logic puzzles, and anything requiring sustained reasoning chains. A model might get a calculation right by recognising a familiar pattern and get a slight variation of it wrong because it is not actually performing the steps. This is why models can ace standardised tests in some domains and fail at simple tasks in others.

Fine-Tuning and Distillation

Once a base model exists, there are ways to make it better for specific tasks without training from scratch. Two of the most important are fine-tuning and distillation.

Fine-tuning means taking a pre-trained model and training it further on a smaller, task-specific dataset. If you want a model that is good at answering customer support tickets, you fine-tune it on thousands of example ticket-and-response pairs. The model already knows language and general knowledge from pre-training; fine-tuning just nudges it toward the patterns, vocabulary, and format your use case needs.

This is far cheaper than training from scratch. You are starting from a model that already works and making targeted adjustments. A full pre-training run might cost millions of dollars in compute; a fine-tuning run can cost a few hundred. This is also why open-weight models like Llama are valuable: anyone can fine-tune them for their own domain without needing the resources to build a foundation model.

Distillation is about efficiency. A large model, say 70 billion parameters, might be very capable but too expensive to run for every query. Distillation trains a smaller model, say 7 billion parameters, to mimic the larger one's outputs. The smaller model will not match the large one on everything, but for many tasks it can get close enough while being an order of magnitude cheaper and faster to run.

The way it works is straightforward in concept: you generate outputs from the large model, then train the small model on those input-output pairs. The small model learns to approximate the large model's behaviour rather than learning from scratch. The result is a model that is cheaper to deploy, faster to serve, and often good enough for production use.

Fine-tuning	Distillation
Adapts a model to a specific task.Pre-trained model + task-specific data produces a model specialised for your use case. Cheaper than training from scratch. Keeps the base model's general knowledge while learning new patterns.	Compresses a model for efficiency.Large model produces a smaller model that approximates it. Trades a small amount of capability for a large reduction in cost and latency. Essential for deploying models at scale on limited hardware.

Model Families Compared

The major LLM families share the same Transformer foundation but differ in how they are trained, what data they use, how they are aligned, and whether their weights are openly available. The table below summarises the key distinctions as of mid-2026.

Model Family	Developer	Access	Key Characteristic
GPT (GPT-4o, o-series)	OpenAI	Closed (API only)	Strong general capability, multimodal (text, image, audio). The o-series introduces reasoning-focused models that think before answering.
Claude	Anthropic	Closed (API only)	Known for careful, nuanced responses and strong refusal of harmful outputs. Emphasis on alignment and honesty. Large context windows.
Gemini	Google DeepMind	Closed (API + consumer products)	Natively multimodal, trained on text, images, audio, and video from the start. Integrated across Google products. Very large context windows.
Llama	Meta	Open weights	Weights released publicly for research and commercial use. Enables fine-tuning and on-device deployment. Spawns many community variants.

The closed versus open distinction matters a lot in practice. With GPT, Claude, and Gemini, you interact with the model through an API and you cannot see or modify the weights. With Llama, you can download the model, fine-tune it on your own data, run it on your own hardware, and audit its behaviour. That freedom comes with the cost of hosting and maintaining the model yourself.

The reasoning-focused models, like OpenAI's o-series and Claude's extended-thinking variants, represent a newer development. These models are trained to spend more compute at inference time, producing intermediate reasoning steps before giving an answer. They tend to be stronger on maths, coding, and multi-step problems, but slower and more expensive per query.

No single model is best for everything. Claude might be better for careful writing, GPT for general versatility, Gemini for multimodal tasks involving images, and a fine-tuned Llama for cost-sensitive deployments where you need control. The right choice depends on the task, the budget, and the constraints.

The field is moving fast

Model capabilities change every few months. A comparison that is accurate today may be outdated by the time you read this. The underlying concepts in this guide, what a language model is, how attention works, why training happens in stages, are more stable than the leaderboard. If you understand the fundamentals, you can evaluate any new model that comes along.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, and agents. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is a large language model and how does it work?

A large language model is an AI system trained on vast amounts of text to predict the next token in a sequence. It breaks text into tokens, learns patterns from training data, and generates responses by continuing the pattern. LLMs power tools like ChatGPT, Claude, and Pascal AI's research agents.

What is the difference between a token and a word in LLMs?

A token is a chunk of text that the model processes. It can be a whole word, a part of a word, or even a single character. English words typically average 1.3 tokens. Tokenization

affects how much text fits in a context window and how much it costs to run a query.

How does Pascal AI use large language models for finance?

Pascal AI uses large language models as the reasoning engine inside its research agents. The models read filings, extract financial data, and generate research outputs. Pascal wraps the model with finance-specific context, citation trails, and governance controls so outputs are auditable and traceable to source documents.

What are the main problems with large language models?

LLMs can hallucinate facts, produce inconsistent outputs, struggle with numerical reasoning, and carry biases from training data. They also have context window limits that constrain how much text they can process at once. These limitations are why human verification is essential for any finance output.